Project Nexelis – Intern-to-Engineer Bootcamp

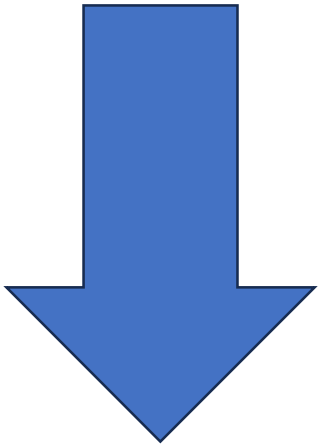
Day 2 – Git & Version Control

Prepared by: Shivam Jha

Role: Software Engineering Intern

Day: 02

Table of Contents:-



1. Objective & Scope

The main objective of Day 2 was to understand how Git and GitHub are used in a real development workflow. I worked on a small Python project and followed the process from creating a repository and feature branch to committing, pushing, reviewing, and preparing the changes for merging.

During the task, I practiced:

- Repository creation and cloning
- Feature branch creation and switching
- Staging and meaningful commits
- Push, pull and fetch operations
- Pull Request and code review process
- Merge and merge-conflict handling
- .gitignore and README documentation

All the practical work was carried out in the intern-engineering-foundation repository using the feature/day2-profile branch.

2. Repository & Branch Setup

I started the Day 2 task by setting up the GitHub repository and connecting it to my local development environment.

Repository:-

intern-engineering-foundation

The repository was initially empty, so I cloned it locally:

git clone <repository-url>

cd intern-engineering-foundation

I then used git status to verify the repository state before starting development.

Creating the Feature Branch:-

To keep the development work separate from the stable branch, I created:

git switch -c feature/day2-profile

All Day 2 development was then carried out on this feature branch.

Branch Structure

main

└── feature/day2-profile

3. Project Structure & Implementation

After setting up the repository and feature branch, I created the project structure specified for the Day 2 task.

Project Structure

Folder / File	Purpose
src/	Contains the Python implementation
tests/	Contains profile tests
docs/	Contains Git workflow documentation
README.md	Project overview and setup information
.gitignore	Excludes unnecessary files from Git

4. Git Operations & Commit History

Once the project files were ready, I followed the Git workflow to track and manage each change.

Git Operations

Operation	Command Used	Purpose
Check status	git status	Check current changes
Stage changes	git add .	Prepare changes for commit

Operation	Command Used	Purpose
Commit	git commit -m "message"	Save a meaningful change
View history	git log --oneline	Review commit history
Push	git push -u origin feature/day2-profile	Upload feature branch to GitHub
Pull	git pull origin feature/day2-profile	Bring remote changes locally
Fetch	git fetch origin	Check remote updates without merging
Merge	git merge <branch>	Combine branch changes

5. Pull Request, Code Review & Merge

After completing the development work and pushing the feature branch to GitHub, I prepared the changes for integration into the main branch.

Pull Request

The Pull Request was created from:

feature/day2-profile → main

The PR provided a single place to review the changes before merging them into the main branch.

Code Review

As part of the PR process, the changes were checked for:

- Correct project structure
- Profile implementation
- Test files
- README and documentation
- Meaningful commit history
- Unnecessary or unwanted files

Merge

After the review stage, the approved feature changes can be merged into main.

The overall workflow followed was:

Develop

↓

Commit

↓

Push

↓

Pull Request

↓

Code Review

↓

Merge

↓

Main

6. Documentation & Final Outcome

Documentation

As part of the Day 2 work, I completed the required project documentation:

README.md:-
Provides the project overview, structure, setup instructions, testing command, and development branch information.

.gitignore:-
Configured to exclude unnecessary files such as Python cache files, virtual environments, and local environment files.

docs/git-workflow.md:-
Documents the Git workflow followed during development, including branching, staging, commits, push, pull, Pull Request, and merge.